Wi-Fi Controlled Robot Car

Project Review 2 / 21MEO106T - Introduction to Robotics

ESP32 with a local browser interface

Architecture, implementation and verification

Saransh Sudan RA2411003011302
Yuvraj Singh Rana RA2411003011300
Deepak M RA2411003011309
Mahanth V RA2411003011621

B.Tech CSE
SRM Institute of Science and Technology
Kattankulathur

Presentation date: awaiting confirmation

Project scope

The Review-1 design uses ESP32 Wi-Fi, an L298N and four geared DC motors

Local connection	The ESP32 creates the RoboCar access point and serves the control page.
Movement	Forward, reverse, independent turning and combined steering commands.
Review-2 changes	Correct input handling, explicit arming, stale-command rejection and timeout stop.
Current boundary	Full-power direction control. No camera, obstacle detection or wheel feedback.

System architecture

A local command path with an independent motor-control task



HTTP: validated command, session token and sequence number

Motor task: checks command age every 10 ms and applies direction outputs

Battery powers the motor rail. An appropriate regulated supply powers the ESP32. Grounds are common.

Signal pin diagram

Default firmware preserves the pin assignment in Review 1

ESP32 GPIO		L298N control	Motor outputs
GPIO 26	→	IN1	OUT1 / OUT2 Left motor pair
GPIO 27	→	IN2	
GPIO 25	→	ENA	OUT3 / OUT4 Right motor pair
GPIO 14	→	IN3	
GPIO 12	→	IN4	
GPIO 15	→	ENB	

Common GND between ESP32, L298N and power supply

GPIO12 / 15 are boot-strapping pins. Optional rewiring uses GPIO13 / 33.

Modules and motor commands

Two driver channels control the left and right sides

Input	Command	Left side	Right side
W / forward	F	Forward	Forward
S / reverse	B	Reverse	Reverse
A / left	L	Reverse	Forward
D / right	R	Forward	Reverse
Release / stop	S	Disabled	Disabled

W+A / W+D: forward arcs. S+A / S+D: reverse arcs. Opposing keys stop.

Control corrections

Behavioral fixes address the failure cases in the original source

Turning from rest	A or D now produces a turn without a preceding forward command.
Combined inputs	Releasing A while W remains held resumes forward motion.
Release and focus	Pointer capture handles releases outside a button. Focus loss disarms.
Communication loss	A 600 ms lease timeout disables the motors and requires rearming.
Late requests	STOP revokes the session token. Duplicate or stale sequences are rejected.

Code and command processing

Small state transitions make behavior explainable in a viva

Firmware timeout

```
if (token && now - lastSeen >= 600) {
   stop();
   ++timeoutCount;
}
```

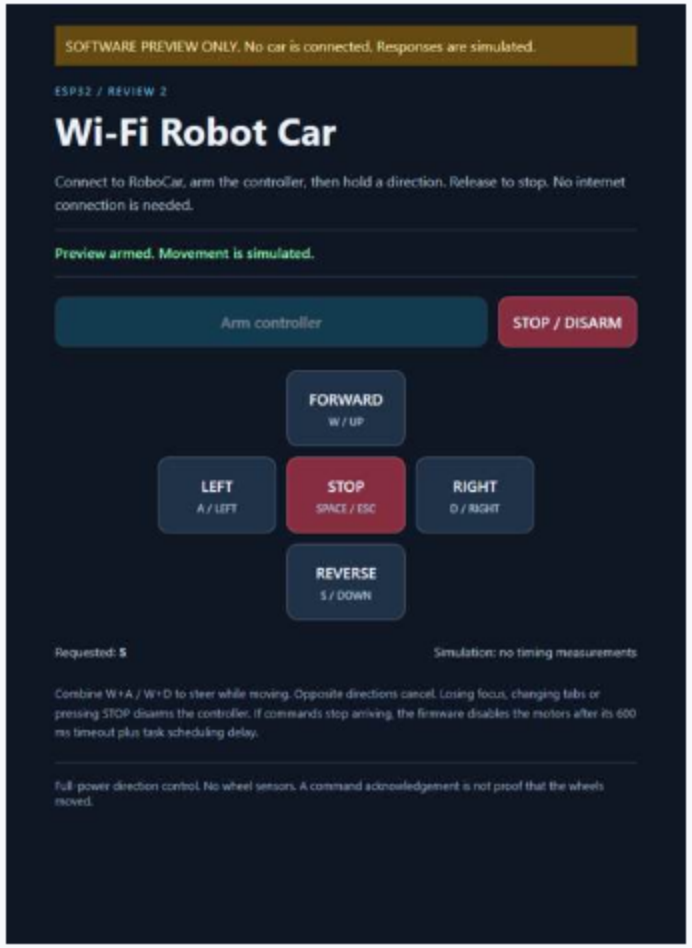
Browser heartbeat

```
setInterval(pump, 150);
```

Each accepted request updates lastSeen. Invalid owners and stale sequences do not renew the lease.

Partial software output

Actual screenshot of the software preview



Browser preview tested in Chrome

Combined keys, release handling and STOP behave as expected in the software preview.

No physical car is connected in this screenshot. Real movement and timing results remain pending.

Software verification

Automated checks and physical tests answer different questions

Check	Current status	Evidence
Browser behavior	9 checks passed	tests/ui.test.cjs
C++ motor state machine	Passed (desktop)	tests/control_core_test.cpp
ESP32 firmware compilation	Passed	962,923 bytes / core 3.3.0
Physical movement / timeout	Not measured	Capture on the actual car

No claim of physical reliability follows from compilation or simulated browser responses.

Literature review

Related work establishes the context for the implementation

Source	Reported approach	Relation to this project
Kamal (2021) [1]	Arduino Uno, ESP8266 and a camera car	Reference for Wi-Fi control architecture
Badiger et al. (2025) [2]	ESP32 camera car with browser control and arm	Broader functionality than this build
Pal et al. (2025) [3]	ESP32, L298N and ultrasonic obstacle stopping	Obstacle sensing is future work here

Our focus: input behavior, communication-loss handling and reproducible evaluation.

Dataset and preprocessing

Experimental command records replace a training dataset

Inputs	Keyboard state and captured touch-pointer state.
Processing	Resolve conflicting inputs, encode command, validate token and sequence.
Test records	Distance, command, expected action, observed action, success and timing.
Data cleaning	Keep failed trials. Mark missing values. Separate HTTP and motion delays.
Current dataset	Collection template prepared. No physical measurements supplied.

Evaluation formulae

Measure the quantity named in each result

Command success	Successful physical trials / attempted physical trials x 100%
Mean HTTP round trip	Sum of request-to-response times / number of valid responses
Motion response	Time of first wheel motion minus time of user input
Vehicle speed	Travel distance / elapsed travel time
Timeout delay	Time of enable LOW minus last accepted command time

Hardware integration

Integration checks remain open until the physical build is inspected

Power path	Confirm battery voltage, motor rating and a suitable ESP32 regulator.
Motor driver	Verify enable jumpers, common ground and current of two motors per side.
Initial test	Raise the wheels. Check boot-stop, direction and STOP before floor tests.
Fault tests	Disconnect Wi-Fi, close the tab and reverse direction under observation.
Evidence	Build photo, labelled wiring photo, Serial Monitor capture and movement video.

GitHub demonstration

Repository contents are prepared for upload

README	Project description, setup and confirmed limitations.
Firmware	RobotCar.ino, Config.h, ControlCore.h and generated WebPage.h.
Tests	Browser checks, C++ controller checks and a compile workflow.
Review documents	Architecture, wiring, literature, experiment protocol and paper draft.
Repository status	Upload pending. No GitHub URL or account was supplied.

Completion tracking

Evidence-weighted checklist, not an estimate of physical assembly

Milestone group	Weight	Status
Architecture and pin documentation	10%	Prepared
Firmware and browser implementation	25%	Prepared
Automated tests and ESP32 build	15%	Passed
Review deck, literature and paper draft	20%	Prepared
Physical integration and experiments	25%	Awaiting evidence
GitHub publication	5%	Awaiting upload

70% of weighted milestones verified. Hardware trials and GitHub upload remain.

Draft paper

An implementation paper with an explicit experimental plan

Working title

Design and Verification of an ESP32 Wi-Fi Robot Car with Browser Control and Command Timeout

Draft content	Abstract, related work, architecture, implementation and test methodology.
Results boundary	Software evidence only until hardware measurements are collected.
Publication plan	Select a suitable venue after results. Verify current submission and indexing details.

Class activities

Only completed session tasks should appear as evidence

Record	What must be attached
Session / task	Exact task assigned by Anirban Sir
Team contribution	Who completed which part
Artifact	Code, circuit, worksheet or screenshot
Learning outcome	What the activity demonstrated
Verification	Completion date and feedback if available

Assigned activities were not supplied. The activity log is ready for actual entries.

Demonstration sequence

A short review walkthrough with observable outcomes

1. Inspect and connect	Raise wheels, power on and connect to the RoboCar network.
2. Show source	Open the GitHub repository and explain one input-to-motor path.
3. Demonstrate movement	Arm. Show forward, reverse, turning from rest and combined-key release.
4. Demonstrate a fault	Stop heartbeats and observe electrical disable and physical coast.
5. Present evidence	Show real logs, test records and the remaining milestones.

Viva preparation

Explain decisions and their limitations

Why ESP32?	It combines the Wi-Fi interface, embedded HTTP server and GPIO control.
Why a motor driver?	GPIO signals control the bridge. The bridge supplies motor current.
Why a timeout?	A release command can be lost. Age-based expiry supplies a fallback.
Why no training data?	This controller maps commands to outputs without machine learning.
What remains open?	Power validation, motor-current suitability and physical performance.

References

Primary technical documentation and related implementations

[1] Kamal, M. A. (2021). Arduino Controlled Spy Robo Car using Wireless Camera With live Streaming. arXiv:2108.03402.

[2] Badiger, P. et al. (2025). WiFi-Controlled Surveillance Car With Web Interface And Robotic Arm Integration. IJCRT, 13(5), l216-l224.

[3] Pal, L., Kumar, P. and Kumar, P. (2025). Design and Development of a Wi-Fi Controlled Robotic Vehicle with Ultrasonic Obstacle Detection. Spectrum of Emerging Sciences, 5(2), 31-36.

[4] Espressif. Arduino-ESP32 Wi-Fi API and ESP32 GPIO documentation.

[5] STMicroelectronics. L298 dual full-bridge driver datasheet.